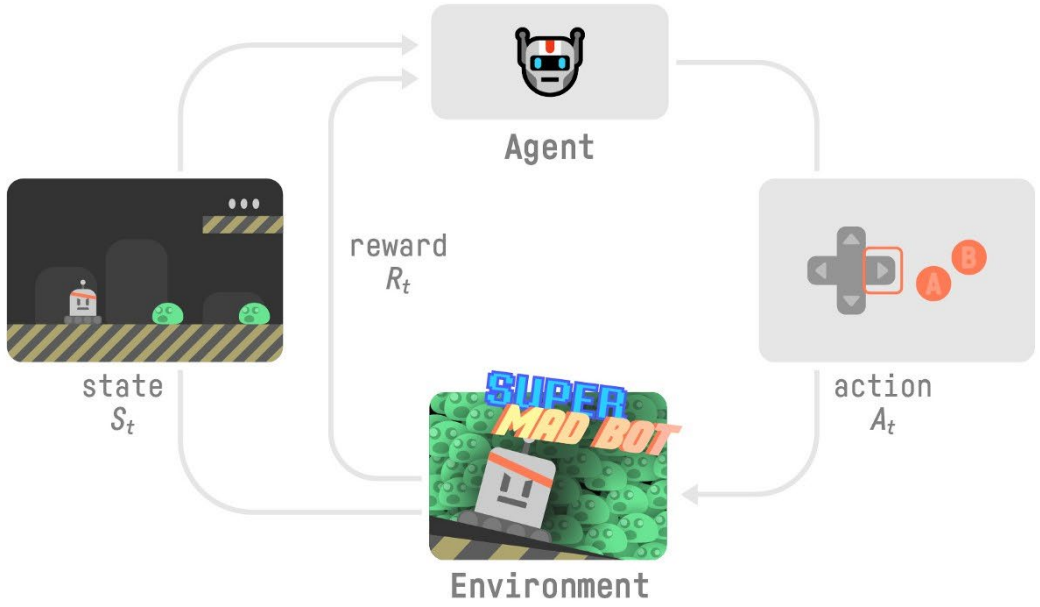
Reinforcement learning

alberto.suarez@uam.es
Computer Science Department
Universidad Autónoma de Madrid

Learning agents

- Sensors to determine the state of the world.
- Actuators to execute actions that change the state of the world.
- Reward function: Function the agent wants to maximize.



Reinforcement learning: Learn by trial and error

Image source: https://huggingface.co/learn/deep-rl-course/unit1/rl-framework

Markov decision process (MDP)

Actions depend only on the current state of the world, not on the history.

- The interaction of the agent with the world is modelled as an interleaved sequence of observation and action steps.

Actions: Discrete. e.g., chess
Continuous. e.g., driving

States: Discrete. e.g., chess
Continuous. e.g., pole balancing

- At each step in the sequence
  - Observation: The agent receives information of the state of the world.
  - Action: The agent takes an action
  - Response: The state of the world changes
  - Reinforcement: The agent receives a reward

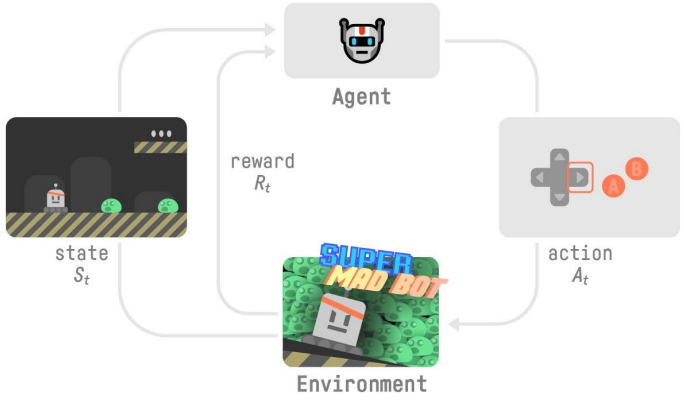
World state
- Completely observed; e.g., chess
- Partially observed; e.g., Mars explorer

$$s_0 \xrightarrow{a_0} r_0, s_1 \xrightarrow{a_1} r_1, s_2 \xrightarrow{a_2} \quad \dots \quad s_{T-1} \xrightarrow{a_{T-1}} r_{T-1}, s_T$$

s_0: Initial state

s_T: Terminal state

- State: $s_{t+1} = s_{t+1}(s_t, a_t)$
- Reward: $r_t = r_t(s_t, a_t)$



GOAL: Maximize the expected cumulative reward

Expected cumulative reward

- Trajectory:

$$\tau_{t:}: \quad s_t \xrightarrow{a_t} r_t, s_{t+1} \xrightarrow{a_{t+1}} r_{t+1}, s_{t+2}, \xrightarrow{a_{t+2}} \quad \dots$$

- **Episodic task**: there are terminal states.
  E.g., chess
- **Continuing taks**: no terminal states. In principle, infinite.
  The task can be terminated by the agent.
  E.g., driving

Cumulative reward:

$$R(\tau_{t:}) = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k} = r_t + \gamma \, r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

Discount rate: $\gamma \leq 1$
If $\gamma < 1$, there is more certainty about short-term rewards.

Policies

Policy-based learning:
Learn $\boldsymbol{\pi}^*$, the policy that yields the highest expected reward.

Policy: $\boldsymbol{\pi} \stackrel{\text{def}}{=} \{\pi_t\}_{t \geq 0}$

- **Deterministic policy**: Function that determines the action to be taken by the agent when at state s_t at time t.

$$a_t = \pi_t(s_t), \qquad t = 0, 1, 2, \ldots$$

$a_t \in A_t(s)$ set of actions that can be taken by the agent when at state s at time t.

$s_t \in S_t$: set of states at which the agent can be at time t.

- **Probabilistic policy**: Probabilities of taking an action

$$\pi_t \stackrel{\text{def}}{=} \{\pi_t(a_t|s_t), \quad a_t \in A_t(s_t)\}, \quad t = 0, 1, 2,$$

$$\sum_{a_t \in A_t(s_t)} \pi_t(a_t|s_t) = 1; \quad s_t \in S_t$$

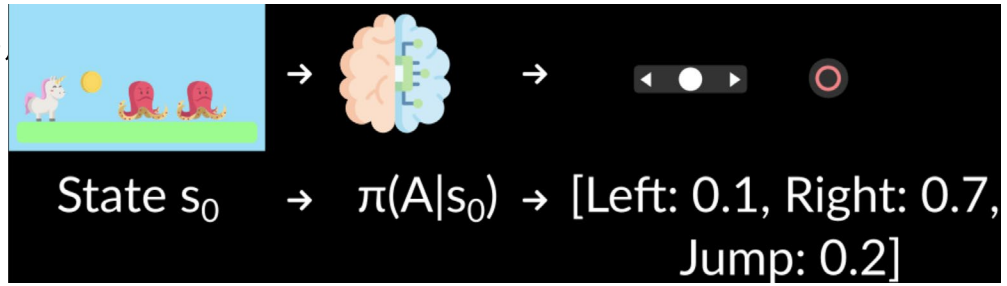


Image source: https://huggingface.co/learn/deep-rl-course/unit1/rl-framework

State value functions

- The value of a state is given by the expected accumulated return of a sequence of actions selected acording to the policy $\boldsymbol{\pi}$ given that the agent is at state s at time t.

$$\mathbf{V}_{\boldsymbol{\pi}} \stackrel{\text{def}}{=} \{V_{\boldsymbol{\pi},t}(s_t); s_t \in S_t\}_{t\geq 0}$$
$$V_{\boldsymbol{\pi},t}(s_t) = \mathbb{E}_{\boldsymbol{\pi}}\left[R(\tau_{t:})\middle|s_t\right]; \quad s_t \in S_t$$

Value-based learning:
Learn $\mathbf{V}_{\boldsymbol{\pi}^*}$ the value functions that yield the expected reward of $\boldsymbol{\pi}^*$, the optimal policy.
Optimal policy (deterministic):
$$V_t^*(s_t) \stackrel{\text{def}}{=} V_{\boldsymbol{\pi}^*,t}(s_t) = \max_{\boldsymbol{\pi}} V_{\boldsymbol{\pi},t}(s_t); s_t \in S_t$$

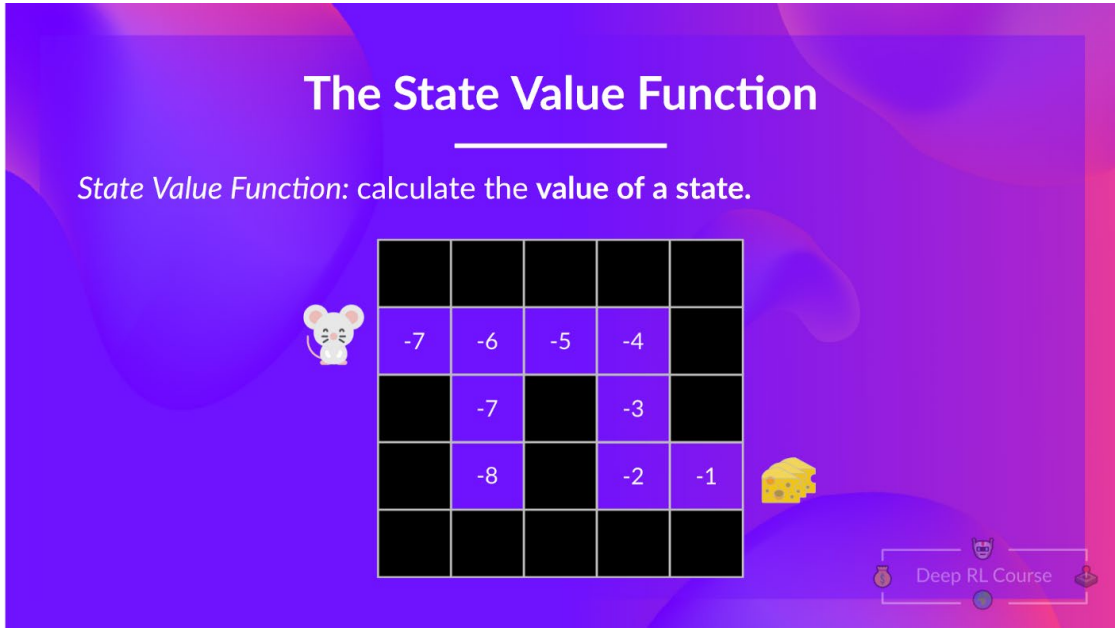


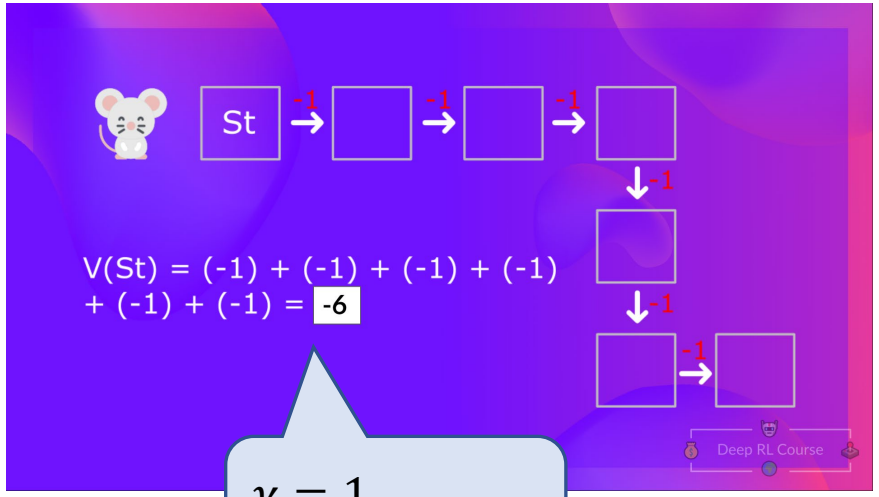
Image source: https://huggingface.co/learn/deep-rl-course/unit1/rl-framework

Bellman equation

Bellman optimality:

$$V_t^*(s_t) = \max_{a_t \in A_t(s_t)} \sum_{s_{t+1}} P_t(s_{t+1}|s_t, a)\big(r_t(s_t, a_t) + \gamma V_{t+1}^*(s_{t+1})\big)$$

$$V_{\boldsymbol{\pi},t}(s_t) = \sum_{a_t \in A_t(s_t)} \pi_t(a_t|s_t) \sum_{s_{t+1}} P_t(s_{t+1}|s_t, a_t)\Big(r_t(s_t, a_t) + \gamma V_{\boldsymbol{\pi},t+1}(s_{t+1})\Big); \quad \text{for all } s_t \in S_t$$



$\gamma = 1$
$r_t(s, a) = -1$

<u>Proof:</u>

$$V_{\boldsymbol{\pi},t}(s_t) = \mathbb{E}_{\boldsymbol{\pi}}[R(\tau_{t:})|s] = \mathbb{E}_{\boldsymbol{\pi}}\big[r_t(s_t, a_t) + \gamma\, R(\tau_{t+1:})\,|s_t\big]$$

$$= \mathbb{E}_{\boldsymbol{\pi}}\big[r_t(s_t, a_t)|s_t\big] + \gamma\, \mathbb{E}_{\boldsymbol{\pi}}\big[R(\tau_{t+1:})\,|s_t\big]$$

$$= \textstyle\sum_{a \in A_t(s)} \pi_t(a_t|s_t) r_t(a_t, s_t)$$

$$+ \gamma \textstyle\sum_{a_t \in A_t(s_t)} \pi_t(a_t|s_t) \sum_{s_{t+1}} P_t(s_{t+1}|s_t, a)\, \mathbb{E}_{\boldsymbol{\pi}}\big[R(\tau_{t+1:})\,|s_{t+1}\big]$$

$$= \textstyle\sum_{a_t \in A_t(s_t)} \pi_t(a_t|s_t) \sum_{s_{t+1}} P_t(s_{t+1}|s_t, a)\Big(r_t(s_t, a_t) + \gamma V_{\boldsymbol{\pi},t+1}(s_{t+1})\Big)$$

Image source: https://huggingface.co/learn/deep-rl-course/unit1/rl-framework

Monte Carlo learning

$$R(\tau_{t:T}) = \sum_{k=0}^{T-1-t} \gamma^k r_{t+k} = r_t + \sum_{k=1}^{T-1-t} \gamma^k r_{t+k}$$
$$= r_t + \sum_{k=0}^{T-1-t-1} \gamma^{k+1} r_{t+k+1}$$
$$= r_t + \gamma \sum_{k=0}^{T-1-(t+1)} \gamma^k r_{(t+1)+k}$$
$$= r_t + \gamma\, R(\tau_{t+1:T})$$

INPUT s_0: initial state

$\boldsymbol{\pi}_{\mathbf{V}}$: stochastic policy based on $\mathbf{V}$ (e.g., ε–greedy)

T: máximum length of episode

γ: discount factor

α: learning rate

OUTPUT: Estimate of $\mathbf{V}_{\boldsymbol{\pi}^*}$

PSEUDOCODE:

$\mathbf{V} \leftarrow \mathbf{0}$ # Initialization for all states (arbitrary)

repeat # Episode loop

Generate from s_0, according to $\boldsymbol{\pi}_{\mathbf{V}}$, an episode of maximum length T

$$\tau_{0:T}\colon\ s_0 \xrightarrow{a_0} r_0, s_1 \xrightarrow{a_1} r_1, s_2 \xrightarrow{a_2} \quad \dots \quad s_{T-1} \xrightarrow{a_{T-1}} r_{T-1}, s_T$$

for t **in** $(T, T-1, \dots, 2, 1, 0)$

$R(\tau_{t:T}) \leftarrow r_t + \gamma R(\tau_{t+1:T})$ # Compute the return function

$V(s_t) \leftarrow V(s_t) + \alpha\left(R(\tau_{t:T}) - V(s_t)\right)$ # Update the value function

until convergence

return $\mathbf{V}$

TD-learning

INPUT s_0: initial state
$\boldsymbol{\pi}_{\mathbf{V}}$: stochastic policy based on $\mathbf{V}$ (e.g., ε–greedy)
T: máximum length of episode
γ: discount factor
α: learning rate

OUTPUT: Estimate of $\mathbf{V}_{\boldsymbol{\pi}^*}$

PSEUDOCODE:

$\mathbf{V} \leftarrow \mathbf{0}$ # Initialization for all states (arbitrary)
repeat # Episode loop
  For t from 0 to $T-1$:
    $s_{t+1} \leftarrow$ generate from s_t successor and reward using $\boldsymbol{\pi}_{\mathbf{V}}$
    $V(s_t) \leftarrow V(s_t) + \alpha\left((r_t + \gamma V(s_{t+1})) - V(s_t)\right)$ # Update value function
    if s_{t+1} is terminal, **then break**
until convergence
return $\mathbf{V}$

TD-learning target

State action value functions

- The value of a state is given by the expected accumulated return of a sequence of actions selected acording to the policy $\boldsymbol{\pi}$ given that the agent is at state s at time t.

Q is for quality

$$\mathbf{Q}_{\boldsymbol{\pi}} \stackrel{\text{def}}{=} \left\{Q_{\boldsymbol{\pi},t}(s_t, a_t); s_t \in S_t; a_t \in A_t(s_t)\right\}_{t \geq 0}$$

$$Q_{\boldsymbol{\pi},t}(s_t, a_t) = \mathbb{E}_{\boldsymbol{\pi}}\left[R(\tau_{t:}) \middle| s_t, a_t\right]$$

GOAL: Learn $\mathbf{Q}_{\boldsymbol{\pi}^*}$, the action-value functions that yiels the expected reward $Q_{\boldsymbol{\pi}^*,t}(s_t, a_t)$ of the agent taking the action $a_t \in A_t(s_t)$ at state $s_t \in S_t$.

Optimal policy (deterministic):

$$\pi_t^*(s_t) = \underset{a \in A_t(s_t)}{\arg\max}\, Q_{\boldsymbol{\pi}^*,t}(s_t, a_t)\,; s_t \in S_t$$

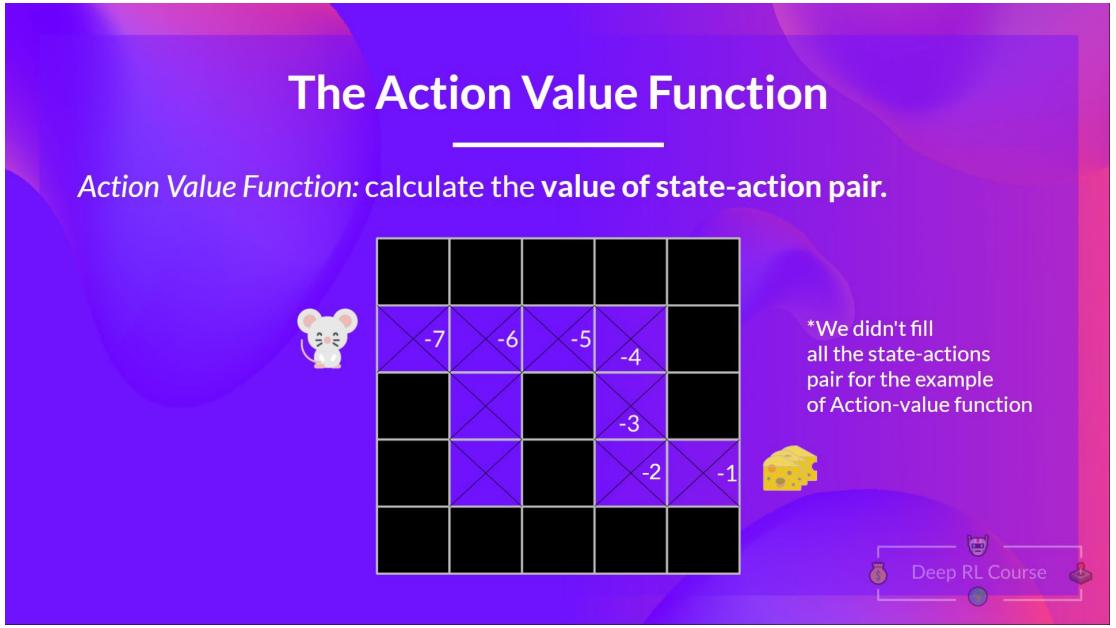


Image source: https://huggingface.co/learn/deep-rl-course/unit1/rl-framework

Bellman equation: action value function

$$Q_{\boldsymbol{\pi},t}(s_t, a_t) = \sum_{s_{t+1}} P_t(s_{t+1}|s_t, a_t)\left(r_t(s_t, a_t) + \gamma \sum_{a_{t+1} \in A_{t+1}(s_t+1)} \pi_t(a_{t+1}|s_{t+1})Q_{\boldsymbol{\pi},t+1}(s_{t+1}, a_{t+1})\right);$$

for all $s_t \in S_t, \quad a_t \in A_t(s_t)$

Bellman optimality:

$$Q_t^*(s_t, a_t) = \max_{a_t \in A_t(s_t)} \sum_{s_{t+1}} P_t(s_{t+1}|s_t, a_t)\left(r_t(s_t, a_t) + \gamma \max_{a_{t+1} \in A_{t+1}(s_{t+1})} Q_{t+1}^*(s_{t+1}, a_{t+1})\right)$$

SARSA: on-policy learning of the Q function

SARSA: State-Action-Reward-State-Action

More stable convergence
Slower convergence (favors exploration)

INPUT s_0: initial state

$\boldsymbol{\pi}_{\boldsymbol{Q}}$: stochastic policy for acting based on $\mathbf{Q}$ (e.g., ε–greedy)

T: maximum length of episode

γ: discount factor

α: learning rate

OUTPUT: Estimate of $\mathbf{Q}_{\boldsymbol{\pi}^*}$

PSEUDOCODE:

$\mathbf{Q} \leftarrow \mathbf{0}$ # Initialization for all states and actions (arbitrary)

$a_0 \leftarrow$ Select action from $A_0(s_0)$ using $\boldsymbol{\pi}_{\mathbf{Q}}$

repeat # Episode loop

  for t **from** 0 **to** $T-1$:

    $r_t, s_{t+1}, \leftarrow$ Take action a_t at state s_t and observe reward and successor

    $a_{t+1} \leftarrow$ Select action from $A_{t+1}(s_{t+1})$ using $\boldsymbol{\pi}_{\mathbf{Q}}$

    # Update the action value function

    $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\left((r_t + \gamma \quad Q(s_{t+1}, a_{t+1})) - Q(s_t, a_t)\right)$

    if s_{t+1} is terminal **then break**

until convergence

return $\mathbf{Q}$

On-policy: Uses $\boldsymbol{\pi}_{\boldsymbol{Q}}$ for both acting and updating (training)

Q-learning: SARSAmax updates (off-policy)

Faster convergence (favors exploitation)
More unstable in stochastic environments

INPUT s_0: initial state
$\boldsymbol{\pi}_{\boldsymbol{Q}}$: stochastic policy for acting based on $\mathbf{Q}$ (e.g., ε–greedy)
T: maximum length of episode
γ: discount factor
α: learning rate

OUTPUT: Estimate of $\mathbf{Q}_{\boldsymbol{\pi}^*}$

PSEUDOCODE:

$\mathbf{Q} \leftarrow \mathbf{0}$ # Initialization for all states and actions
repeat
  for t **from** 0 **to** $T-1$:
    $a_t \leftarrow$ Select action from $A_t(s_t)$ using $\boldsymbol{\pi}_{\mathbf{Q}}$
    $r_t, s_{t+1}, \leftarrow$ Take action a_t at state s_t and observe reward and successor state
    # Update action value function
    $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left((r_t + \gamma \max_{a_{t+1} \in A_{t+1}(s_{t+1})} Q(s_{t+1}, a_{t+1})) - Q(s_t, a_t) \right)$
    if s_{t+1} is terminal **then break**
until convergence
return $\mathbf{Q}$

Off-policy: Uses greedy policy (not $\boldsymbol{\pi}_{\boldsymbol{Q}}$) for updating (training)

Exploration vs exploitation tradeoff

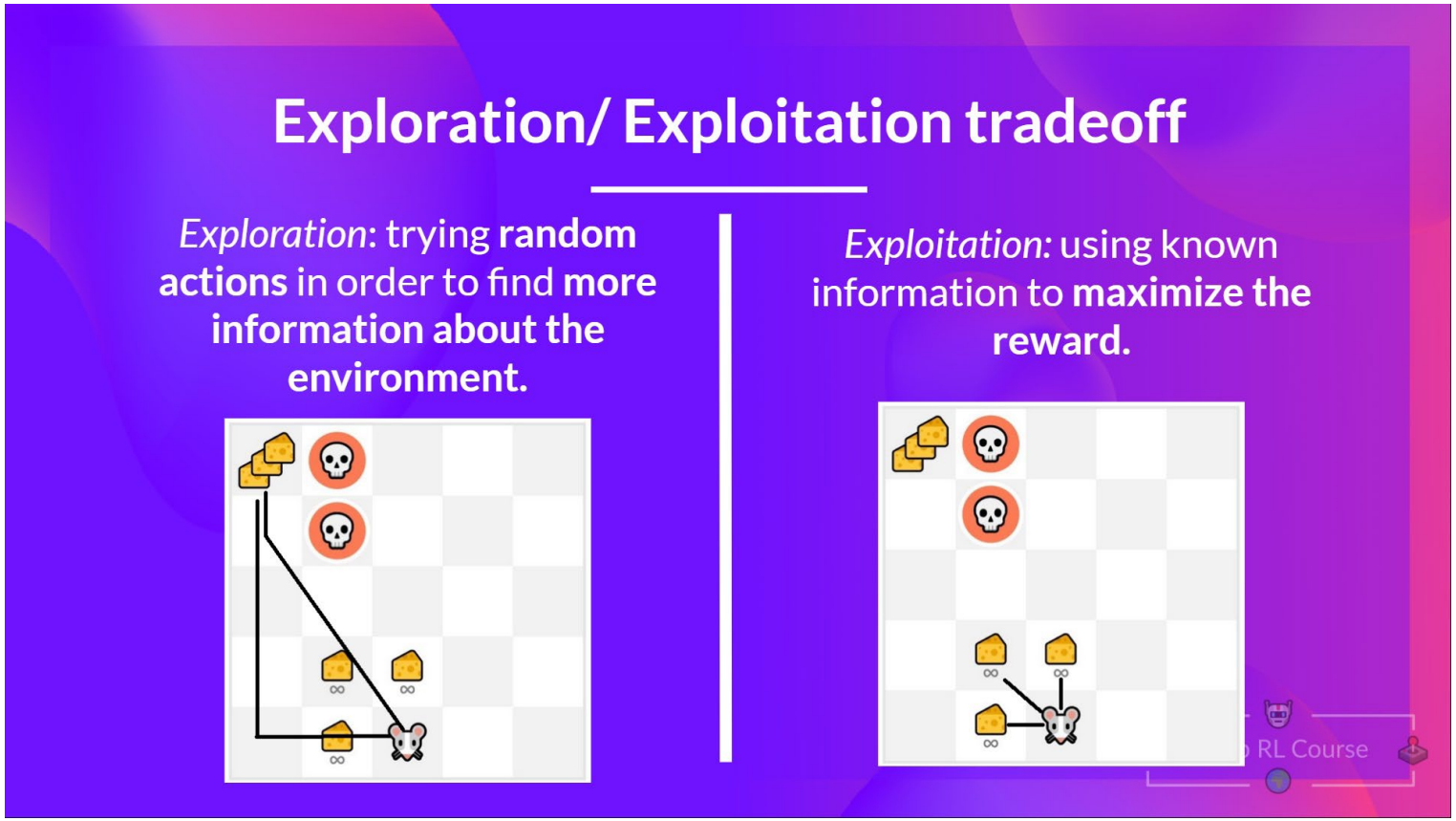


Image source: https://huggingface.co/learn/deep-rl-course/unit1/rl-framework

ε-greedy exploration strategy

The ε-greedy policy of a reinforcement learning agent at state s_t at time t based on $\mathbf{Q}$ is

$$a_t = \begin{cases} \text{random action in } A_t(s_t) & \text{with probability} \quad \varepsilon_t \\ \arg\max_{a \in A_t(s_t)} Q(s_t, a) & \text{with probability} \quad 1 - \varepsilon_t \end{cases}$$

Choices for ε_t

- Constant: $\varepsilon_t = \varepsilon < 1$, typically small

e.g., $\varepsilon = 0.1$

- Decaying:
  - Algebraic decay: $\varepsilon_t = \frac{\varepsilon_0}{t}$
  - Exponential decay: $\varepsilon_t = \varepsilon_{min} + (\varepsilon_{max} - \varepsilon_{min})e^{-ct}$

More exploration at the beginning
More exploitation at the end

Standard vs deep Q-learning

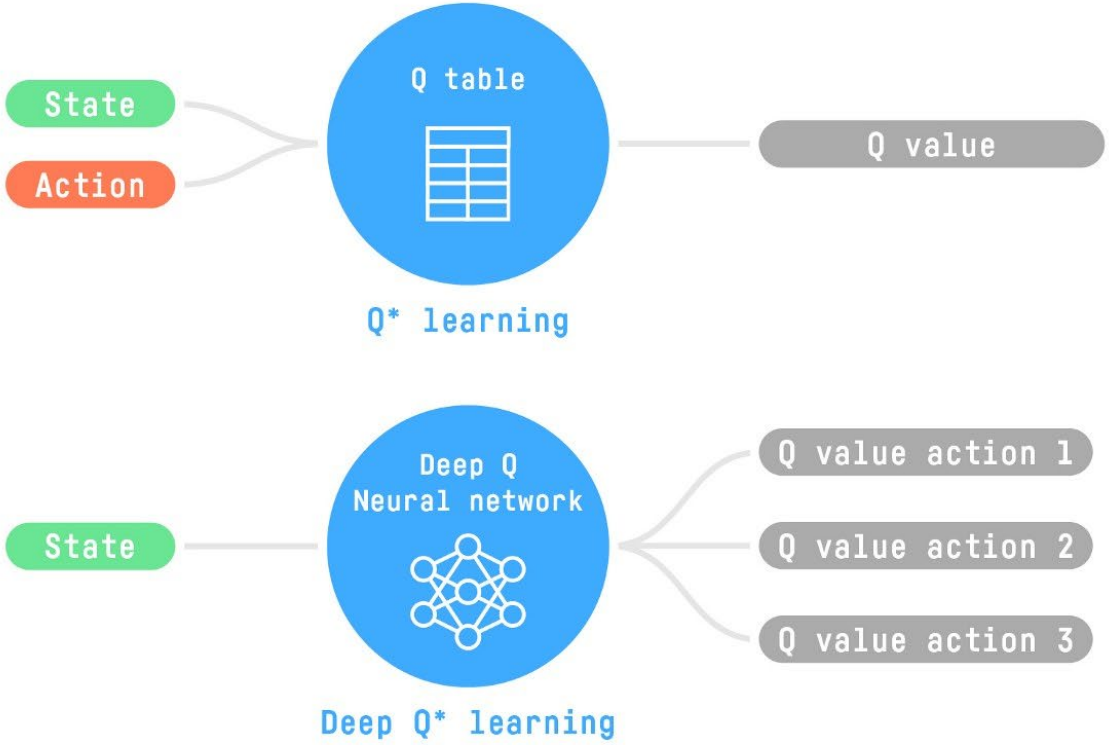


Image source: https://huggingface.co/learn/deep-rl-course/unit1/rl-framework

Policy–based learning

GOAL: Learn the optimal policy directly.

$$\pi_t^*(s_t) = \underset{a \in A_t(s_t)}{\arg\max}\, Q_{\boldsymbol{\pi}^*,t}(s_t, a_t)\,;\, s_t \in S_t$$

METHOD:

- Consider a parametric family of probabilistic policies $\boldsymbol{\pi}_{\boldsymbol{\theta}}(a_t|s_t)$
- Optimize the parameters of the policy

e.g., a neural network

T: Time horizon (can be infinite)

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\arg\max}\, J(\boldsymbol{\theta}) = \underset{\boldsymbol{\theta}}{\arg\max}\, \mathbb{E}_{\boldsymbol{\pi}_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T-1} \gamma^t R_t \,\middle|\, S_0 = s_0\right]$$

Policy–based learning: pole balancing

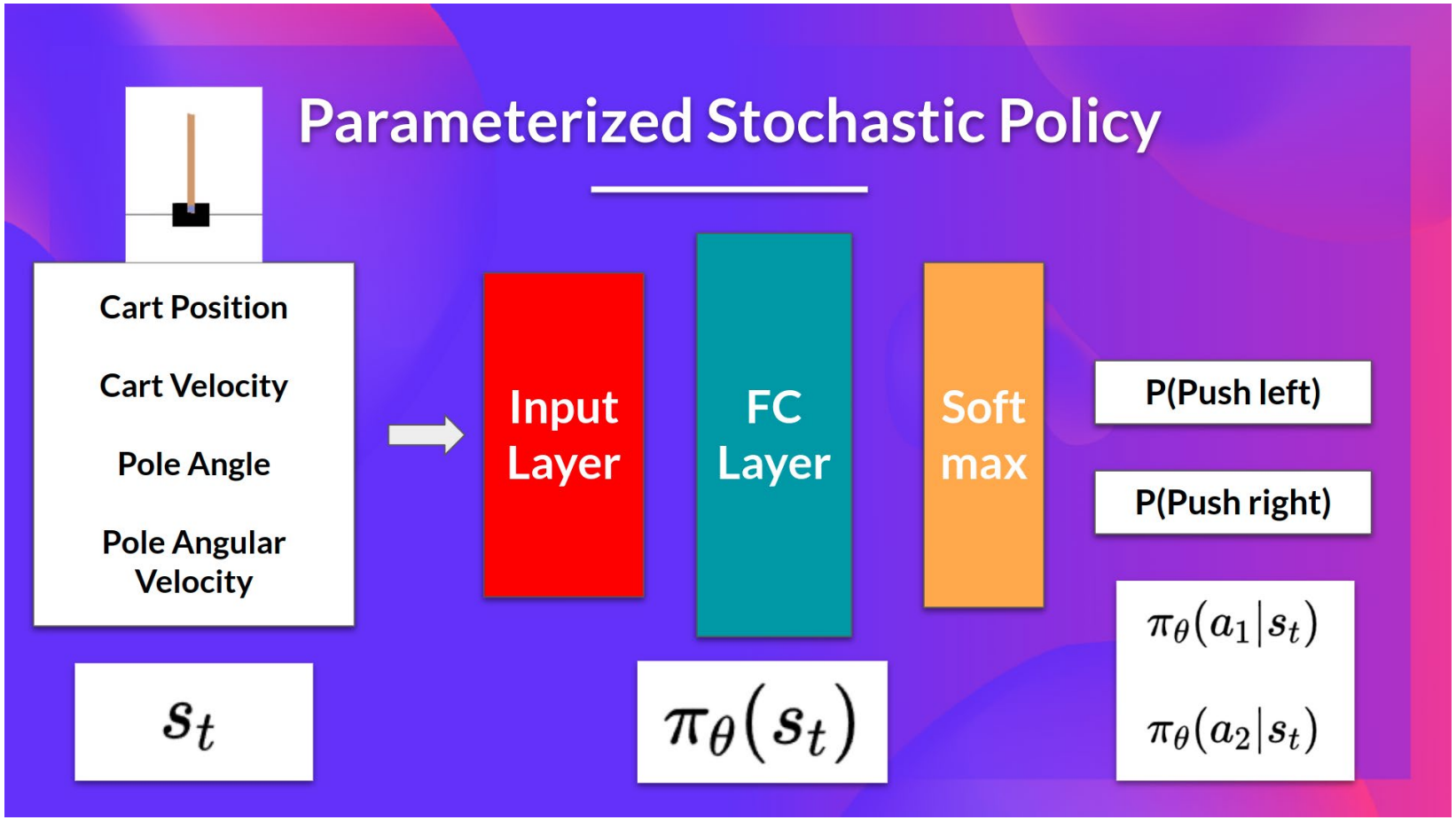


Image source: https://huggingface.co/learn/deep-rl-course/unit4/what-are-policy-based-methods

Policy gradient

- Objective function

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{\theta}}\left[\sum_{t=0}^{T-1} \gamma^t R_t \,\middle|\, S_0 = s_0\right]$$

- Gradient of the objective function

$\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t)$: Direction in parameter space along which the probability of taking action a_t at state s_t is increased

Cumulative reward

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{\pi}_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T-1} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A_t|S_t) \sum_{t=0}^{T-1} (\gamma^t R_t) \,\middle|\, S_0 = s_0\right]$$

Accumulated reward

$$= \mathbb{E}_{\boldsymbol{\pi}_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T-1} \left(\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A_t|S_t) \sum_{\tau=t}^{T-1} (\gamma^\tau R_\tau)\right) \,\middle|\, S_0 = s_0\right]$$

Policy learning: The REINFORCE algorithm

INPUT s_0: initial state
$\boldsymbol{\theta}_0$: Intial value of the parameters of the policy network.
N: Number of trajectories per paramtr updat
T: maximum length of episode
γ: discount factor
α: learning rate

OUTPUT: Estimate of $\mathbf{Q}_{\boldsymbol{\pi}^*}$

PSEUDOCODE:

$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_0$ # Initialization of the weights of the policy network
repeat
  for n **from** 1 **to** N:
    for t **from** 0 **to** $T-1$:
      $a_t^{(n)} \leftarrow$ Select action from $A_t\left(s_t^{(n)}\right)$ using $\boldsymbol{\pi}_{\boldsymbol{\theta}}$
      $r_t^{(n)}$, $s_{t+1}^{(n)} \leftarrow$ Take action $a_t^{(n)}$ at state $s_t^{(n)}$ and observe reward and successor state
      if $s_{t+1}^{(n)}$ is terminal **then break**
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \frac{1}{N} \sum_{n=1}^{N} \left[\sum_{t=0}^{T-1} \left(\nabla_{\boldsymbol{\theta}} \log \boldsymbol{\pi}_{\boldsymbol{\theta}} \left(a_t^{(n)} \middle| s_t^{(n)} \right) \sum_{\tau=t}^{T-1} \left(\gamma^{\tau-1} r_\tau^{(n)} \right) \right) \right]$
until convergence
return $\boldsymbol{\pi}_{\boldsymbol{\theta}}$

Monte Carlo estimate of the gradient

Ronald J. Williams. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. Mach. Learn. 8, 3–4 (May 1992), 229–256. https://doi.org/10.1007/BF00992696